

Formulación de la Solución — Problema de los 8 Caballos

Estado Inicial

El tablero se representa como una matriz `int[8][8]` inicializada en cero, donde:

- `0` → casilla vacía
- `1` → casilla ocupada por un caballo

Caso 1 - Tablero vacío:

Todas las celdas son `0`. El algoritmo inicia con `caballosColocados = 0` y busca la primera posición segura desde `(0, 0)`.

Caso 2 - Caballo preubicado:

El usuario indica una posición `(fila, columna)`. Esa celda se marca con `1` antes de llamar al algoritmo, que inicia con `caballosColocados = 1`.

```
java
// Tablero vacío (Caso 1)
tablero = new int[8][8]; // todos los valores = 0

// Caballo preubicado en (2, 3) (Caso 2)
tablero[2][3] = 1;
ColocarCaballos(1);
```

Decisiones Posibles

En cada llamada recursiva, el algoritmo recorre **todas las celdas** del tablero en orden fila-columna (de izquierda a derecha, de arriba hacia abajo) y decide:

┃ ¿Es seguro colocar un caballo en la celda `(fila, columna)`?

Si la respuesta es **sí**, coloca el caballo y recursa. Si la respuesta es **no**, prueba la siguiente celda.

```
java
```

```

for (int fila = 0; fila < 8; fila++) {
    for (int columna = 0; columna < 8; columna++) {
        if (esSegura(fila, columna)) {
            tablero[fila][columna] = 1;           // decisión: colocar
            if (ColocarCaballos(caballosColocados + 1)) return true;
            tablero[fila][columna] = 0;           // retroceso: quitar
        }
    }
}

```

Validaciones

Se aplican dos niveles de validación:

1. Validación de entrada (método `ColocarCaballos`)

Garantiza que el número de caballos recibido sea coherente con el problema:

```

java

if (caballosColocados < 0 || caballosColocados > 8)
    throw new IllegalArgumentException("El número de caballos debe estar entre 0 y 8")

```

2. Validación de posición segura (método `esSegura`)

Una celda `(fila, columna)` es **segura** si cumple ambas condiciones:

Condición	Verificación en el código
La celda está vacía	<code>tablero[fila][columna] == 0</code>
Ningún caballo ya colocado la ataca	Se comprueban los 8 movimientos en <code>L</code> posibles

```

java

```

```
// Para cada uno de los 8 movimientos posibles del caballo:
int filaAtaque = fila + movimiento[0];
int columnaAtaque = columna + movimiento[1];

// Solo se evalúa si la celda atacante está dentro del tablero
if (filaAtaque >= 0 && filaAtaque < 8 &&
    columnaAtaque >= 0 && columnaAtaque < 8) {
    if (tablero[filaAtaque][columnaAtaque] == 1)
        return false; // hay conflicto
}
```

Los 8 deltas que representan el movimiento en L del caballo son:

```
(-2,-1) (-2,+1)
(-1,-2) (-1,+2)
(+1,-2) (+1,+2)
(+2,-1) (+2,+1)
```

Caso Base

La recursión termina exitosamente cuando ya se han colocado los 8 caballos:

```
java

if (caballosColocados == 8) {
    return true; // solución encontrada
}
```

En este punto el tablero contiene una configuración válida y no se realiza ninguna acción adicional. La condición se evalúa **antes** de intentar colocar otro caballo, lo que garantiza que nunca se intente colocar un noveno.

Paso de Retroceso (Backtracking)

Si después de colocar un caballo en `((fila, columna))` ninguna de las siguientes posiciones permite completar la solución, el algoritmo **deshace** la decisión:

```
java
```

```

tablero[fila][columna] = 1;                                // 1. Colocar caballo

if (ColocarCaballos(caballosColocados + 1)) {             // 2. Continuar recursión
    return true;
}

tablero[fila][columna] = 0;                                // 3. RETROCESO: liberar celda

```

El retroceso restaura la celda a $\textcircled{0}$, permitiendo que la iteración continúe probando otras celdas. Si se agotan todas las celdas sin encontrar solución, el método retorna `false`, propagando el retroceso al nivel anterior de la recursión.

Flujo general del backtracking

```

ColocarCaballos(0)
├─ prueba (0,0) → segura → coloca
│   └─ ColocarCaballos(1)
│       └─ prueba (0,0) → ocupada → salta
│       └─ prueba (0,1) → ¿atacada? → ...
│           └─ ColocarCaballos(2)
│               └─ ...
│                   └─ ColocarCaballos(8) → ✓ CASO BASE → true
│   ← si falla todo → retroceso: libera (0,1)
├─ si falla todo → retroceso: libera (0,0)
└─ prueba (0,1) → ...

```